Chapitre2: Les structure de contrôle

Arfaoui Ahlem

Les structures de contrôle en C

Alternative: if-else
Choix Multiple: switch-case
Itérations: for, while, do-while

1) Les structures de contrôle:

1.1) les structures conditionnelles ou alternatives:

- **If - else:**

- Syntaxe

Pas de **then** en C

Le bloc " **else** " est optionnel

```
If(condition)
{
Instruction 1;
.....
}
else
{
Instruction 2;
.....
}
```

```
If (a<b)
{
Min=a;
}
else
{
Min=b;
}
```

- **Si** la<condition> fournit une valeur différente de zéro, **alors** le <**bloc d'instructions 1**> est exécuté
- **Si** la<condition> fournit la valeur zéro, **alors** le <**bloc d'instructions 2**> est exécuté
- Tout ce qui est **0** ('\0' 0 0.0000 NULL) est **faux**
- Tout ce qui est **!= de 0** (1 '0' 0.0001 1.34) est **vrai**
- **Exemple:**

```
if(32)
printf("ceci sera toujours affiche\n");
if(0)
printf("ceci ne sera jamais affiche\n");
```

La partie **<condition>** peut désigner :
- une variable d'un type numérique,
- une expression fournissant un résultat numérique.

La partie **<bloc d'instructions>** peut désigner :
- un (vrai) bloc d'instructions compris entre accolades,
- une seule instruction terminée par un point-virgule.

```
If(condition)
{
Instruction 1;
.....
}
else
{
Instruction 2;
.....
}
```

Exemple 1:

```
if (EGAL)
printf("A est égal à B\n");
else
printf("A est différent de B\n");
```

Exemple 2: Permutation.

```
if (A > B)
{
AIDE = A;
A = C;
C = AIDE;
}
else
{
AIDE = B;
B = C;
C = AIDE;
}
```

- **else** est associé avec le **if** le plus proche.

```
Int i=100;
If(i<0)
  if(i>100)
     printf(‘’i>1000\n’’);
  else
      printf(‘’i est raisonnable\n ’’);
```

I est raisonnable

```
Int i=-100;
If(i<0){
   if(i>100)
      printf(‘’i>1000\n’’);
 }
else
    printf(‘’i est négative\n ’’);
```

I est négative

- **if - else if - ... – else:**

En combinant plusieurs structures **if - else** en une expression nous obtenons une structure qui est très courante pour prendre des décisions entre plusieurs alternatives:

```
if ( <cond1> )
<instruction 1>
else if (<cond2>)
< instruction 2>
else if (<cond3>)
< instruction 3>
else if (<condN>)
< instruction N>
else
< instruction N+1>
```

* Les conditions <cond1> ... <condN> sont évaluées **du haut vers le bas** jusqu'à ce que l'une d'elles soit différente de zéro. Le bloc d'instructions y lié est alors exécuté et le traitement de la commande est terminé.

Exemple:

```
#include <stdio.h>
main()
{
int A,B;
printf("Entrez deux nombres entiers :");
scanf("%i %i", &A, &B);
if (A > B)
printf("%i est plus grand que %i\n", A, B);
else if (A < B)
printf("%i est plus petit que %i\n", A, B);
else
printf("%i est égal à %i\n", A, B);
return 0;
}
```

La dernière partie **else** traite le cas où aucune des conditions n'a été remplie. Elle est **optionnelle**, mais elle peut être utilisée très confortablement pour détecter des erreurs.

Exemple:

Développer un programme C qui permet de résoudre l'équation ax + b = 0.

Les opérateurs conditionnels:

Le langage C possède **une paire d'opérateurs** qui peut être utilisée comme alternative à **if - else** et qui a l'avantage de pouvoir être intégrée dans une expression.

Syntaxe:

<expr1> ? <expr2> : <expr3>

- Si **<expr1>** fournit une valeur différente de zéro, alors la valeur de **<expr2>** est fournie comme résultat
- Si **<expr1>** fournit la valeur zéro, alors la valeur de **<expr3>** est fournie comme résultat

Exemple:

La suite d'instructions

```
if (A>B)
MAX=A;
else
MAX=B;
```

peut être remplacée par:



```
MAX = (A > B) ? A : B;
```

1.2) Structure à choix multiple: Switch

*** syntaxe:**

```
Switch(expression)
{
Case valeur1: instruction1;
break;
Case valeur2: instruction2;
break;
……..
Case valeurN: instructionN;
break;
Default:  Autre_instruction;
}
```

Exemple:



```
include<stdio.h>
main() {
 float i,j;
char op;
printf("donnez votre opération : +, -, * ou / ");
scanf("%c", &op);
printf("donnez le premier nombre ");
scanf("%f", &i);
printf("donnez le deuxième nombre ");
scanf("%f", &j);
double r=0;
switch(op) {
case '+': { r=i+j; printf("%f", r); break; }
case '-': printf("%f",i-j); break;
case '*': printf("%f", i*j); break;
case '/': { //printf("La division est %f \n",i/j);
 if (j!=0) { printf("La division est %f \n",i/j); }
 else {printf("operation impossible");} break; }
default: printf(" pas d'opération ");}
 }
```

* Chaque <**instruction**> suivie éventuellement de **break**; pour arrêter l'exécution des autres cas. Les <**valeur**> peuvent être constante ou expression constante entière ou caractère. En tout cas elles doivent être différentes.
L'expression est évaluée et est comparée à toutes les constantes. Dès que sa valeur est égale à une de ces constantes, l'exécution reprend au choix correspondant.

- **Default** est optionnel et est exécuté quand aucun des **case** ne correspond à l'expression qui apparaît dans le **switch (expression).** C'est généralement une bonne idée d'y mettre le signalement d'une erreur.

Remarque: les instructions break qui apparaissement après les instructions de chaque **case** sont primordiales**.** En effet, en leur absence, l'exécution se poursuivrait avec les instructions du case suivant. Par exemple, si expression est égale à valeur2 et que le **break** situé à la fin des instructions du **case valeur 2:** est omise, les instructions du **case valeur 3:** seront exécutées à la suite.
* Le nombre de **case** n'est pas limité.

Exemple:
Ecrire un programme qui traduit un nombre entre 1 et 12 en le mois de l'année qui le correspond.

```
int mois;
scanf (˝%d˝ , &mois);
switch (mois)
{
 case 1: printf (˝ janvier ˝); break;
 case 2: printf (˝ fevrier ˝); break;
…
case 12: printf (˝ décembre ˝); break;
default: printf (˝ erreur ˝);
}
```

1.2) les structures répétitives ou itératives:

Il existe trois sortes de structures répétitives**: while**, **do while** et **for.**

- **While =** tant que:

Syntaxe:

```
 while(condition)
{
bloc d'instructions ;
}
```

=> tant que condition est vraie (≠0) on exécute bloc d'instructions. Si la condition fournit la valeur zéro, l'exécution continue avec l'instruction qui suit le bloc d'instructions.

Remarques:

- S'il n'y a qu'une seule instruction, les accolades sont inutiles.
- Le bloc d'instructions est exécuté 0 ou plusieurs fois.

La partie <condition> peut désigner :

•une variable d'un type numérique,

•une expression fournissant un résultat numérique.

Exemple:

```
int i = 0;
while (i<4)
{
printf("%i \n", i++);
}
```

Affichage:



```
0
1
2
3
```

Exemple:
int i =4;
while(i)
printf("%d\n",i--);

Affichage:



4
3
2
1

Remarque:

Parfois nous voulons seulement attendre un certain événement, sans avoir besoin d'un traitement de données. Dans ce cas, la partie <bloc d'instructions> peut être vide (notation: **;** ou **{}**).

La ligne suivante ignore tous les espaces entrés au clavier et peut être utilisée avant de lire le premier caractère significatif:

while (getch()==' ‘)
;

*** Do-while**

La structure **do - while** est semblable à la structure **while**, avec la différence suivante :

* **while** évalue la condition avant d'exécuter le bloc d'instructions,

* **do - while** évalue la condition après avoir exécuté le bloc d'instructions. Ainsi le bloc d'instructions est exécuté au moins une fois.

Syntaxe:

do
{
bloc d'instructions ;
} **while**(condition) ; => on exécute bloc d'instructions tant que condition est vrai (≠0)

Remarques :

- S'il n'y a qu'une seule instruction, les accolades sont inutiles.
- Le bloc d'instructions est exécuté au moins une fois, et aussi longtemps que condition fournit une valeur vraie (différente de 0).

En pratique, **do-while** est moins courant que **while**, mais fournit parfois une solution plus élégante.

Exemple:

```
int n ;
 do {
printf("Entrer un nombre entre 1 et 10:\n") ;
scanf("%d",&n);
 } while(n<1||n>10) ;
```

Exemple:

```
int n, div ;
printf("Entrer le nombre à diviser:\n") ;
scanf("%d",&n);
do {
printf("Entrer le diviseur:\n") ;
scanf("%d",&div);
} while(!div) ;
 printf("%d/%d = %f\n",n,div,(float)n/div) ;
```

- **For:**

La structure **for** en C est plus générale et beaucoup plus puissante.
syntaxe:
for(**expr1**;**expr2**;**expr3**)
{ bloc d'instructions ; }

expr1 est évaluée une fois avant le passage dans la boucle, elle est utilisée pour initialiser les données de la boucle.
expr2 est évaluée à chaque passage de la boucle, elle est utilisée pour savoir si la boucle est répétée ou non (c'est une condition de répétition, et non d'arrêt).
expr3 est évaluée à la fin de chaque passage de la boucle, elle est utilisée pour réinitialiser les données de la boucle.

Equivalence entre une boucle **for** et une boucle **while**:
expr1;
while(**expr2**){
bloc d'instructions ;
expr3; }

Le plus souvent, for est utilisé comme boucle de comptage:
for(**init;cond_repetition;compteur**)
{ bloc d'instructions ; }

Exemple:

```
int i;
for(i=1;i<=4;i++)
{
printf("Le carré de %d est %d\n",i,i*i);
 }
```



Affichage:
Le carré de 1 est 1
Le carré de 2 est 4
Le carré de 3 est 9
Le carré de 4 est 16

Exemple:

```
int i;
for(i=4;i>0;i--)
{
printf("Le carré de %d est %d\n",i,i*i);
}
```



Affichage:
Le carré de 4 est 16
Le carré de 3 est 9
Le carré de 2 est 4
Le carré de 1 est 1

Choix de la structure répétitive:

On choisit la structure qui reflète le mieux l'idée du programme que l'on veut réaliser, en respectant les directives suivantes:

- Si le bloc d'instructions ne doit pas être exécuté si la condition est fausse -> **while** ou **for**
- Si le bloc d'instructions doit être exécuté au moins une fois -> **do-while**

-Si le nombre d'exécution du bloc d'instructions dépend de une ou plusieurs variables qui sont modifiées à la fin de chaque répétition -> **for**

-Si le bloc d'instructions doit être exécuté aussi longtemps qu'une condition extérieure est vraie (par exemple aussi longtemps qu'il y a des données dans un fichier d'entrées) -> **while**

Exercice:
Ecrivez un programme qui lit N nombres entiers au clavier et qui affiche leur somme, leur produit et leur moyenne.
Choisissez un type approprié pour les valeurs à afficher. Le nombre N est à entrer au clavier. Résolvez ce problème,
a) en utilisant **while,**
b) en utilisant **do - while**,
c) en utilisant **for**.
d) Laquelle des trois variantes est la plus naturelle pour ce problème?